

AI ASSISTED CODING

Lab Assignment-4.3

2303A51463

B-07

Task 1: Zero-Shot Prompting – Leap Year Check

[1]

def is_leap_year(year):
 if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
 return True
 else:
 return False

Example usage:
print(f"2000 is a leap year: {is_leap_year(2000)}")
print(f"1900 is a leap year: {is_leap_year(1900)}")
print(f"2024 is a leap year: {is_leap_year(2024)}")
print(f"2023 is a leap year: {is_leap_year(2023)}")

...

2000 is a leap year: True
1900 is a leap year: False
2024 is a leap year: True
2023 is a leap year: False

Release notes

Gemini X

Zero-Shot Prompt Used Generate a Python function that accepts a year as input, checks whether the given year is a leap year, and returns the result.

Here's a Python function that checks if a year is a leap year, along with an example of how to use it:

The code has executed successfully, and the `is_leap_year` function correctly identified:

- 2000 as a leap year (True)
- 1900 as not a leap year (False)
- 2024 as a leap year (True)
- 2023 as not a leap year (False)

This confirms the function is working as expected based on the rules for leap years.

[2]

print(is_leap_year(2024))

True

[3]

Os

print(is_leap_year(2023))

False

Task 2: One-Shot Prompting – Centimeters to Inches Conversion

Release notes

Gemini X



G Convert centimeters to inches using Python. Example: Input: 10 cm → Output: 3.94 inches Now generate a Python function to perform this conversion.

◆ Here's a Python function to convert centimeters to inches, along with an example:

The code has executed successfully. The `cm_to_inches` function correctly converted 10 centimeters to 3.94 inches, as demonstrated by the example output.



[4]
✓ 0s

```
def cm_to_inches(cm):  
    """  
    Converts a measurement from centimeters to inches.  
    """  
    inches = cm / 2.54  
    return inches  
  
# Example usage:  
input_cm = 10  
output_inches = cm_to_inches(input_cm)  
print(f"{input_cm} cm \u2192 {output_inches:.2f} inches")
```

10 cm → 3.94 inches

[5]
✓ 0s

```
print(cm_to_inches(10))
```

3.937007874015748

[6]
✓ 0s

```
print(round(cm_to_inches(10),2))
```

... 3.94



Task 3: Few-Shot Prompting – Name Formatting

Release notes

Gemini X



G Format a full name as “Last, First” using Python. Examples: Input: "John Smith" → Output: "Smith, John" Input: "Anita Rao" → Output: "Rao, Anita" Input: "Rahul Verma" → Output: "Verma, Rahul" Now generate a Python function to perform this formatting.

◆ Certainly! Here's a Python function that formats a full name as "Last, First", along with some examples:

The code has executed successfully. The `format_full_name` function correctly reformatted the names from 'First Last' to 'Last, First' for multi-part names. For a single-part name like 'OnlyOneName', it returned the original name, which is consistent with the function's definition.



[Explain the 'format_full_name' function](#)

[Generate a Python function to validate email addresses](#)

[Test 'format_full_name' with a list of names](#)

```
[8] def format_full_name(full_name):  
  
    name_parts = full_name.split()  
    if len(name_parts) >= 2:  
        first_name = name_parts[0]  
        last_name = name_parts[-1]  
        return f"{last_name}, {first_name}"  
    else:  
        return full_name # Return original name if not enough parts  
  
    # Example usage:  
    print(f'"John Smith" -> '{format_full_name("John Smith")}'')  
    print(f'"Anita Rao" -> '{format_full_name("Anita Rao")}'')  
    print(f'"Rahul Verma" -> '{format_full_name("Rahul Verma")}'')  
    print(f'"OnlyOneName" -> '{format_full_name("OnlyOneName")}'') # Edge case
```

▼ ... 'John Smith' -> 'Smith, John'
'Anita Rao' -> 'Rao, Anita'
'Rahul Verma' -> 'Verma, Rahul'
'OnlyOneName' -> 'OnlyOneName'

```
[10] print(format_full_name("John Smith"))  
✓ Os
```

▼ Smith, John



```
[11] print(format_full_name("Anita Rao"))  
✓ Os
```

▼ ... Rao, Anita

Task 4: Comparative Analysis – Zero-Shot vs Few-Shot

Zero-Shot:

Release notes

Gemini X



G Zero-Shot Prompt Used Generate a Python function that counts the number of vowels in a string. (No examples provided → zero-shot)

✦ Here's a Python function to count the number of vowels in a string, along with an example:

The code has executed successfully. The `count_vowels` function correctly identified the number of vowels in the provided strings:

- 'Hello World' contains 3 vowels.
- 'Python Programming' contains 4 vowels.
- 'Rhythm' contains 0 vowels.



Add more examples for count_vowels function

Explain the code for count_vowels function

[13]
✓ Os

```
def count_vowels(input_string):  
    """  
    Counts the number of vowels (a, e, i, o, u, case-insensitive)  
    in a given string.  
    """  
    vowels = "aeiouAEIOU"  
    count = 0  
    for char in input_string:  
        if char in vowels:  
            count += 1  
    return count  
  
# Example usage:  
text1 = "Hello World"  
text2 = "Python Programming"  
text3 = "Rhythm"  
  
print(f"Number of vowels in '{text1}': {count_vowels(text1)}")  
print(f"Number of vowels in '{text2}': {count_vowels(text2)}")  
print(f"Number of vowels in '{text3}': {count_vowels(text3)}")
```

...
Number of vowels in 'Hello World': 3
Number of vowels in 'Python Programming': 4
Number of vowels in 'Rhythm': 0

Few-Shot:

Release notes

Gemini X



G Count vowels in a string using Python. Examples: Input: "hello" → Output: 2
Input: "AI Tools" → Output: 4 Input: "xyz" → Output: 0 Now generate a Python function.

◆ You already have a `count_vowels` function defined in your notebook. I will update that cell to include the new examples you provided:

The `count_vowels` function in the updated cell has successfully executed, including the new examples.

- For 'hello', it correctly found 2 vowels.
- For 'AI Tools', it correctly found 4 vowels.
- For 'xyz', it correctly found 0 vowels.

All results are as expected.



[Explain how the 'count_vowels' function works](#)

[Add more test cases for 'count_vowels'](#)

[14]

```
def count_vowels(input_string):  
    vowels = "aeiouAEIOU"  
    count = 0  
    for char in input_string:  
        if char in vowels:  
            count += 1  
    return count  
  
# Example usage:  
text1 = "Hello World"  
text2 = "Python Programming"  
text3 = "Rhythm"  
  
print(f"Number of vowels in '{text1}': {count_vowels(text1)}")  
print(f"Number of vowels in '{text2}': {count_vowels(text2)}")  
print(f"Number of vowels in '{text3}': {count_vowels(text3)}")  
print(f"Number of vowels in 'hello': {count_vowels('hello')}")  
print(f"Number of vowels in 'AI Tools': {count_vowels('AI Tools')}")  
print(f"Number of vowels in 'xyz': {count_vowels('xyz')}")
```

...
Number of vowels in 'Hello World': 3
Number of vowels in 'Python Programming': 4
Number of vowels in 'Rhythm': 0
Number of vowels in 'hello': 2
Number of vowels in 'AI Tools': 4
Number of vowels in 'xyz': 0

Comparison Table:

Criteria	Zero-Shot Output	Few-Shot Output
Accuracy	Correct	Correct
Readability	Moderate	High
Logical Clarity	Explicit loop	Concise & clear
Code Length	Longer	Shorter
Efficiency	Good	Better

- **Zero-shot prompting** produces correct but more basic logic
- **Few-shot prompting** improves:
 - Code clarity
 - Conciseness
 - Professional style
- Few-shot prompting is **more effective for pattern-based and logic-refinement tasks**

Task 5: Few-Shot Prompting – File Handling

Release notes

Gemini X

⌵ ⋮ ✕

G

Read a text file and count the number of lines using Python. Examples: Input file with 3 lines → Output: 3 Input file with 5 lines → Output: 5 Now generate a Python function that reads a .txt file and returns the number of lines.

✦

Here's a Python function to count the number of lines in a text file. I'll also include an example that creates a temporary file and then demonstrates the function's usage.

The code has executed successfully, and the `count_lines_in_file` function performed as expected:

- It correctly identified 4 lines in the `example.txt` file that was created.
- It also gracefully handled the case of a `non_existent_file.txt`, returning the appropriate error message, demonstrating robust error handling.

👍 💬

Explain the `'count_lines_in_file'` function in detail

```

def count_lines_in_file(filepath):
    try:
        with open(filepath, 'r') as file:
            line_count = sum(1 for line in file)
            return line_count
    except FileNotFoundError:
        return f"Error: File not found at {filepath}"
    except Exception as e:
        return f"An error occurred: {e}"

dummy_file_content = "This is line 1.\nThis is line 2.\nAnd this is line 3.\nLast line, line 4."
dummy_filepath = 'example.txt'

with open(dummy_filepath, 'w') as f:
    f.write(dummy_file_content)

print(f"Created '{dummy_filepath}' with the following content:\n---\n{dummy_file_content}\n---")
# Count lines in the dummy file
num_lines = count_lines_in_file(dummy_filepath)
print(f"Number of lines in '{dummy_filepath}': {num_lines}")
# Example with a file that doesn't exist
missing_file_path = 'non_existent_file.txt'
num_lines_missing = count_lines_in_file(missing_file_path)
print(f"Number of lines in '{missing_file_path}': {num_lines_missing}")

... Created 'example.txt' with the following content:
---
This is line 1.
This is line 2.
And this is line 3.
Last line, line 4.
---
Number of lines in 'example.txt': 4
Number of lines in 'non_existent_file.txt': Error: File not found at non_existent_file.txt

```

Logic Explanation:

- The file is opened in **read mode**.
- `readlines()` reads all lines into a list.
- `len()` counts the number of lines.
- The function returns the total line count.
- `with open()` ensures the file is closed automatically.